

FILE LOCKING MECHANISM

Pseudocode for Using Read and Write Locks on a File

Step 1: Define Helper Functions for Locking and Unlocking

Initialize Read Lock Function:

Define a function `lock_file_read(fd)`:

Set up a struct flock to specify the lock details.

Set `l_type` to `F_RDLCK` (read lock).

Set `l_whence` to `SEEK_SET` to start from the beginning of the file.

Set `l_start` to 0 and `l_len` to 0 to cover the entire file.

Use `fcntl(fd, F_SETLK, &lock)` to apply the read lock.

If `fcntl` returns -1, check if `errno` is `EACCES` or `EAGAIN`, indicating the file is locked by another write lock.

If locked, print a message and return an error.

If successfully locked, return success.

Initialize Write Lock Function:

Define a function `lock_file_write(fd)`:

Set up a struct flock similar to the read lock, but with `l_type` set to `F_WRLCK` (write lock).

Use `fcntl(fd, F_SETLK, &lock)` to attempt to acquire the write lock.

If `fcntl` returns -1, check for `EACCES` or `EAGAIN`, indicating the file is already locked by another read or write lock.

If locked, print a message and return an error.

If successfully locked, return success.

Define Unlock Function:

Define a function `unlock_file(fd)`:

Set up a struct flock with `l_type` set to `F_UNLCK` to release the lock.

Use `fcntl(fd, F_SETLK, &lock)` to release the lock.

If `fcntl` returns -1, print an error message.

Otherwise, confirm the lock is released.

Step 2: Implement File Access Functions with Locks

Function to Read Data with Read Lock:

Open the file in read mode.

Call `lock_file_read(fd)` to lock the file for reading.

If the file is locked successfully, proceed with reading data.

Read the required data from the file.

Once the read operation is complete, call `unlock_file(fd)` to release the lock.

Close the file after unlocking.

Function to Write Data with Write Lock:

Open the file in write mode.

Call `lock_file_write(fd)` to lock the file for writing.

If the file is locked successfully, proceed with writing data.

Write the required data to the file.

Once the write operation is complete, call `unlock_file(fd)` to release the lock.

Close the file after unlocking.

Step 3: Main Execution Logic

Open the file you want to work with in the appropriate mode (read or write).

Determine if the process needs a read or write lock.

If reading data, call the read function that acquires a read lock and performs the read operation.

If writing data, call the write function that acquires a write lock and performs the write operation.

Ensure proper error handling to detect if the file is already locked by another process.

Always unlock the file after operations are completed, regardless of success or failure, to avoid deadlocks.

Pseudocode

function lock_file_read(fd)

 Initialize a flock structure with l_type = F_RDLCK

 Set l_whence = SEEK_SET, l_start = 0, l_len = 0

 if fcntl(fd, F_SETLK, &lock) == -1 then

 if errno is EACCES or EAGAIN

 Print "Read lock failed, file is locked by another process"

 return ERROR

 else

 Print "Read lock acquired successfully"

 return SUCCESS

function lock_file_write(fd)

 Initialize a flock structure with l_type = F_WRLCK

 Set l_whence = SEEK_SET, l_start = 0, l_len = 0

 if fcntl(fd, F_SETLK, &lock) == -1 then

 if errno is EACCES or EAGAIN

 Print "Write lock failed, file is locked by another process"

 return ERROR

 else

 Print "Write lock acquired successfully"

 return SUCCESS

function unlock_file(fd)

 Initialize a flock structure with l_type = F_UNLCK

 Set l_whence = SEEK_SET, l_start = 0, l_len = 0

 if fcntl(fd, F_SETLK, &lock) == -1

 Print "Error unlocking file"

 return ERROR

else

Print "File unlocked successfully"

return SUCCESS

function read_file(fd)

if lock_file_read(fd) == SUCCESS then

Read data from file

unlock_file(fd)

else

Print "Could not lock file for reading"

function write_file(fd)

if lock_file_write(fd) == SUCCESS then

Write data to file

unlock_file(fd)

else

Print "Could not lock file for writing"